SeedingMinds

Assessment Name

Java Coding_S2_170126

Module: Coding

Marks: +10 -0

Question 1:

Matrix Signal Propagation

You are given an `N x M` matrix `H` where each cell `(r, c)` contains a positive integer `H[r][c]` representing a "processing delay factor".

A signal originates at a source cell `(R_s, C_s)` at time `t = 0`. The signal can travel from any cell to any of its four adjacent cells (up, down, left, right). Each move from one cell to an adjacent one takes exactly `1` unit of time.

However, there's a special rule for propagation: when a signal arrives at a cell `(r, c)` at time `t_arrival`, it incurs a mandatory "processing delay". The signal can only depart from this cell at the earliest time `t_departure` such that:

1. `t_departure >= t_arrival`
2. `t_departure` is a multiple of the cell's delay factor, `H[r][c]`.

The time the signal arrives at the next adjacent cell will be `t_departure + 1`. Your goal is to find the minimum possible time for the signal to arrive at a target cell `(R_t, C_t)`.

Input Format

- The first line contains two space-separated integers, `N` and `M`.
- The second line contains four space-separated integers: `R_s, C_s, R_t, C_t` (0-indexed).
- The next `N` lines each contain `M` space-separated integers, representing the matrix `H`.

Output Format

- Print a single integer representing the minimum arrival time at the target cell. If the target is unreachable, print `-1`.

Constraints:

- `1 <= N, M <= 1000`
- `0 <= R_s, C_s, R_t, C_t < N, M`
- `1 <= H[r][c] <= 100,000`

Example:

Sample Input:

```
3 3
0 0 2 1
2 8 5
4 1 9
7 3 6
```

Sample Output:

```
6
```

Explanation:

NA

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	3 3 0 0 2 1 2 8 5 4 1 9 7 3 6	6
2	2 2 0 0 1 1 1 100 100 1	101
3	1 5 0 0 0 4 1 1 1 1 1	4

Question 2:

Marks: +10 -0

Vowel-Consonant Alternation

Write a program to rearrange a given string `S` based on a specific set of rules. The goal is to create a new string where vowels and consonants from the original string are placed in an alternating sequence.

The rearrangement must follow these rules:

1. The relative order of vowels among themselves and consonants among themselves must be preserved from the original string.
2. The new string should start with the character type (vowel or consonant) of the first character of the original string `S`.
3. After alternating as long as possible, if there are remaining characters in one group (either vowels or consonants), they should be appended to the end of the new string.

For the purpose of this problem, vowels are 'a', 'e', 'i', 'o', 'u' (case-insensitive). All other alphabetic characters are considered consonants.

Input Format:

- A single line containing the string `S`.

Output Format:

- A single line containing the rearranged string.

Constraints:

- `1 <= length of S <= 1000`
- The string `S` will only contain uppercase and lowercase English alphabetic characters (`a-z`, `A-Z`)

Example:

Sample Input:

gemini

Sample Output:

```
gemini
```

Explanation:

NA

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	helloworld	helolowrld
2	aeiou	aeiou
3	Rhythm	Rhythm

Question 3:

Marks: +10 -0

Wildlife Rescue Operation

In a dedicated wildlife conservation center, a team of researchers is tasked with organizing a rescue operation for various endangered species. Each animal has a unique identification number and an associated urgency score based on their condition and the threats they face in the wild. The researchers need to prioritize which animals to rescue first, focusing on those with the highest urgency scores. Your task is to develop a program that collects data about the animals in need of rescue. The program should prompt the user to enter the total number of animals and their respective urgency scores. After collecting the data, the program must sort only the urgency scores while leaving the identification numbers unchanged. Finally, it should output the urgency scores in sorted order and indicate which urgency scores correspond to the animals that were prioritized for rescue.

Constraints:

- The number of animals can be up to 1,000.
- Urgency scores can be any integer value (positive or negative).
- Identification numbers are unique for each animal.

Example:

Example 1:

Input: 101 102 103 104 105
5 2 8 3 7
Output: 103:8, 105:7, 101:5, 104:3, 102:

Example 2:

Input: 801 802 803 804
0 0 0 0
Output: 801:0, 802:0, 803:0, 804:0

Explanation:

Your task is to create a program that efficiently manages and sorts the urgency of animal rescues based on their assigned IDs.Your objective is to sort the animal rescues primarily by their urgency scores in descending order. In cases where multiple animals share the same urgency score, their IDs should be displayed in ascending order. The output should provide a clear listing of each animal ID paired with its urgency score, formatted as ID:Score.

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	901 902 903 904 905 4 2 1 4 1	901:4, 904:4, 902:2, 903:1, 905:1

#	INPUT	EXPECTED OUTPUT
2	1001 1002 1003 15 20 10	1002:20, 1001:15, 1003:10

Question 4:

Marks: +10 -0

Academic Standings Challenge

In a competitive academic environment, students are rigorously evaluated on their performance across multiple subjects. The evaluation isn't simply based on their raw scores, as each student is assigned a rank, which reflects their standing relative to others. The ranks dictate the priority in which their scores should be considered during final grading and analysis.
This system has been designed to give every student a chance to shine, not just based on raw performance but also by giving importance to strategic rankings. As a result, students may have high scores but lower ranks, or lower scores but higher ranks, affecting how their results are evaluated.

Constraints:

- The number of scores (n) will be equal to the number of ranks.
- Each score and rank will be positive integers, with scores in the range of 0 to 10000.

Example:

Example 1:

Input: 10 20 15
1 2 3
1
Output: [15, 10, 20]

Example 2:

Input: 35 25 15 5
1 3 4 2
2
Output: [25, 15, 35, 5]

Explanation:

You have been given the responsibility to automate the process of determining the final standing of students. Your mission is to take the array of scores and their corresponding ranks, sort the scores based on the ranks, and then rotate the sorted list of scores by a given number of positions. This way, the final standing reflects both the rank and the adjustment caused by rotation, which adds an extra layer of evaluation.

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	60 40 80 70 4 2 1 3 3	[40, 70, 60, 80]
2	-15 25 -35 45 4 3 1 2 1	[-15, -35, 45, 25]